

C++ OOP Syntax Cheat Sheet (Γρήγορη Υπενθύμιση)

1. Δομή Κλάσης & Έλεγχος Πρόσβασης (Access Control)

```
class MyClass {
private:
    // Κατά προεπιλογή (default) μέσα σε 'class'.
    // Πρόσβαση μόνο από μεθόδους της MyClass και friend κλάσεις/συναρτήσεις.
    int privateVar;

protected:
    // Πρόσβαση από τη MyClass, τις παράγωγες κλάσεις (derived) και friends.
    int protectedVar;

public:
    // Πρόσβαση από παντού.
    int publicVar;

    // Λίστα Αρχικοποίησης Μελών (Member Initialization List)
    // Υποχρεωτική για const μέλη, αναφορές (references) και κλήση constructor τη
    MyClass(int a, int b) : privateVar(a), protectedVar(b) {}
};

struct MyStruct {
    // Κατά προεπιλογή (default) μέσα σε 'struct', όλα είναι public!
    int publicByDefault;
};
```

2. Μητρώο Πρόσβασης Κληρονομικότητας (Inheritance Access Matrix)

```
class Derived : <AccessSpecifier> Base { ... };
```

Πρόσβαση στη Βασική (Base)	: public Κληρονομικότητα	: protected Κληρονομικότητα	: private Κληρονομικότητα
private	Κρυφό (Καμία άμεση πρόσβαση)	Κρυφό (Καμία άμεση πρόσβαση)	Κρυφό (Καμία άμεση πρόσβαση)
protected	Γίνεται protected στη Derived	Γίνεται protected στη Derived	Γίνεται private στη Derived
public	Γίνεται public στη Derived	Γίνεται protected στη Derived	Γίνεται private στη Derived

3. Σειρά Κατασκευής & Καταστροφής (Construction & Destruction)

- **Κατασκευή (Construction):** Βασικές Κλάσεις (από αριστερά προς τα δεξιά στη δήλωση) → Αντικείμενα-Μέλη (με τη σειρά δήλωσής τους στην κλάση) → Παράγωγη Κλάση (Derived).
- **Καταστροφή (Destruction):** Η ακριβώς αντίστροφη σειρά της Κατασκευής (LIFO - Last In, First Out).

4. Πολυμορφισμός & Εικονικοί Destructors (SOS)

Εικονικές Συναρτήσεις (Virtual Functions)

```
class Base {
public:
    virtual void print() const;           // Εικονική συνάρτηση (δυναμική σύνδεση)
    virtual void pureVirtual() = 0;       // Καθαρή εικονική (Pure Virtual) ->
};

class Derived : public Base {
public:
    void print() const override;          // override: προαιρετικό αλλά προτείνεται
    void pureVirtual() override;          // Υλοποίηση καθαρής εικονικής
};
```

Εικονικοί Καταστροφείς (Virtual Destructors) - Ο Χρυσός Κανόνας

Αν διαγράψουμε δείκτη της Base που δείχνει σε Derived αντικείμενο και ο καταστροφέας της Base **δεν** είναι `virtual`, ο καταστροφέας της Derived **δεν θα κληθεί ποτέ** (Memory Leak!).

```
class Base {
public:
    // ΠΑΝΤΑ virtual destructor σε Base κλάσεις (όταν υπάρχει πολυμορφισμός)!
    virtual ~Base() { /* Αποδέσμευση πόρων Base */ }
};

class Derived : public Base {
public:
    ~Derived() override { /* Αποδέσμευση πόρων Derived */ }
};
```

5. Πολλαπλή Κληρονομικότητα (Πρόβλημα του Διαμαντιού)

```
class Ancestor { /* ... */ };

// Η εικονική κληρονομικότητα εγγυάται ότι η Ancestor θα δημιουργηθεί μόνο ΜΙΑ φορά
class ParentA : virtual public Ancestor { /* ... */ };
class ParentB : virtual public Ancestor { /* ... */ };

class Child : public ParentA, public ParentB {
public:
    Child() : Ancestor(), ParentA(), ParentB() {}
};
```

```
};
```

6. Ορθότητα ως προς Const (Const Correctness)

```
class ConstDemo {
private:
    mutable int accessCounter; // mutable: τροποποιείται ακόμα και σε const μεθόδ
    int data;
public:
    // const μέθοδος: Δεν τροποποιεί μέλη (εκτός από τα mutable).
    // Μπορεί να κληθεί από const αντικείμενα.
    int getData() const {
        accessCounter++;
        return data;
    }
};
```

7. Φίλες Συναρτήσεις & Κλάσεις (Friend Functions/Classes)

```
class Target {
private:
    int secret;
    friend void externalHelper(const Target& t); // Πρόσβαση στα private
    friend class Inspector;                    // Πρόσβαση στα private από όλη
};
```

8. Υπερφόρτωση Τελεστών (Operator Overloading)

8.1 Οδηγός Σύνταξης & Μεθοδολογίας

Για να ορίσεις έναν τελεστή σωστά, απάντησε σε 3 ερωτήσεις:

1. Μέλος (Member) ή Φίλη Συνάρτηση (Friend);

- **Ως Μέλος:** Όταν το αριστερό μέλος είναι αντικείμενο της κλάσης μας (π.χ. `obj1 + obj2`). Το `this` αντιπροσωπεύει το αριστερό μέλος.
- **Ως Friend (Ελεύθερη):** Όταν το αριστερό μέλος είναι άλλος τύπος (π.χ. `cout << obj` ή `double * obj`). Πρέπει να περάσουμε και τα δύο ορίσματα ρητά στην παρένθεση.

2. Τι μπαίνει στην Παρένθεση (Ορίσματα);

- Στις συναρτήσεις-μέλη, το αριστερό μέλος περνάει αυτόματα. Έτσι, για δυαδικούς τελεστές (π.χ. `+`, `-`, `==`) βάζουμε **μόνο 1 παράμετρο** (το δεξί μέλος). Για μοναδιαίους (π.χ. `prefix ++`) βάζουμε **0 παραμέτρους**.

3. Τι Επιστρέφει η Συνάρτηση;

- **Επιστροφή με Αναφορά (`Class&`):** Όταν η πράξη τροποποιεί το αντικείμενο απευθείας (π.χ. `+=`, `=`, `prefix ++`). Επιτρέπει αλυσίδες πράξεων.

- **Επιστροφή με Τιμή (Class)**: Όταν δημιουργείται νέο, προσωρινό αντικείμενο (π.χ. `+`, `-`, postfix `++`).

8.2 Πίνακας Κανόνων Υπερφόρτωσης

Πράξη / Τελεστής	Σύνταξη (Ως Μέλος)	Σύνταξη (Ως Friend)	Επιστροφή	Ορίσματα
Ανάθεση Αντιγραφής (=)	<code>Class& operator= (const Class& other)</code>	Δεν επιτρέπεται ως <i>friend</i>	<code>Class&</code>	<code>(const Class& other)</code>
Μαθηματικοί (+ , -)	<code>Class operator+ (const Class& other) const</code>	<code>friend Class operator+(const Class& a, const Class& b)</code>	<code>Class</code>	<code>(const Class& other)</code>
Διαδικός Πολλαπλασιασμός (*)	<code>Class operator* (double scalar) const</code>	<code>friend Class operator*(double s, const Class& obj)</code>	<code>Class</code>	<code>(double scalar)</code>
Έμμεση Αναφορά / Dereference (*)	<code>T& operator*()</code>	Σπάνια ως <i>friend</i>	<code>T& (Αναφορά)</code>	<code>()</code> [Κενή]
Σύγκριση (==)	<code>bool operator== (const Class& other) const</code>	<code>friend bool operator==(const Class& a, const Class& b)</code>	<code>bool</code>	<code>(const Class& other)</code>
Prefix Αύξηση (++obj)	<code>Class& operator++ ()</code>	<code>friend Class& operator++(Class& obj)</code>	<code>Class&</code>	<code>()</code> [Κενή]
Postfix Αύξηση (obj++)	<code>Class operator++ (int)</code>	<code>friend Class operator++(Class& obj, int)</code>	<code>Class</code>	<code>(int)</code> [Dummy τιμή]
Ροή Εξόδου (<<)	Δεν γίνεται	<code>friend ostream& operator<<(ostream& os, const Class& obj)</code>	<code>ostream&</code>	<code>(ostream& os, const Class& obj)</code>

8.3 Παράδειγμα Υπερφόρτωσης Τελεστών

```
class Vector { private: double x, y; public: Vector(double x = 0, double y = 0) : x(x), y(y) {} // 1. Διαδικός
+ (Επιστρέφει νέο αντικείμενο με τιμή) Vector operator+(const Vector& other) const { return
Vector(x + other.x, y + other.y); } // 2. Ανάθεσης += (Τροποποιεί το αντικείμενο και επιστρέφει
```

αναφορά) `Vector& operator+=(const Vector& other) { x += other.x; y += other.y; return *this; } // 3.`
 Δείκτη [] (Επιστρέφει αναφορά στο μέλος για ανάγνωση/γγραφή) `double& operator[](int index) { return (index == 0) ? x : y; } // 4. Prefix ++v (Αυξάνει και επιστρέφει αναφορά) Vector& operator++() { ++x; ++y; return *this; } // 5. Postfix v++ (Κρατάει αντίγραφο, αυξάνει, επιστρέφει το αντίγραφο) Vector operator++(int) { Vector temp = *this; ++(*this); // Καλεί τον prefix ++ return temp; } // 6. Ροής << (Υποχρεωτικά Friend - Αριστερό μέλος εκτός κλάσης) friend std::ostream& operator<<(std::ostream& os, const Vector& v) { os << "(" << v.x << ", " << v.y << ")"; return os; } //
 Παράδειγμα Μοναδιαίου operator* (Dereference) class SmartPtr { private: Vector* ptr; public: SmartPtr(Vector* p = nullptr) : ptr(p) {} ~SmartPtr() { delete ptr; } // Επιστρέφει αναφορά στο αντικείμενο που «δείχνει» Vector& operator*() { return *ptr; } //`

9. Αντιγραφή Αντικειμένων: Ρηχή vs Βαθιά Αντιγραφή (Deep Copy)

9.1 Θεωρητική Διαφορά στη Μνήμη

Αν έχουμε μια κλάση που διαχειρίζεται έναν δείκτη `char* buffer` :

- **Ρηχή Αντιγραφή (Shallow Copy - Default):**

Αντικείμενο A → Διεύθυνση 0x1A4 ← Αντικείμενο B

- Συνέπεια: Δύο αντικείμενα δείχνουν στην ίδια μνήμη στη Heap. Η τροποποίηση του ενός αλλάζει το άλλο. Κατά την καταστροφή έχουμε **Double Free Error (Crash)**.
- **Βαθιά Αντιγραφή (Deep Copy):**

Αντικείμενο A → Διεύθυνση 0x1A4 (Δεδομένα A)

Αντικείμενο B → Διεύθυνση 0x5B2 (Αυτόνομο Αντίγραφο)

- Συνέπεια: Πλήρης ανεξαρτησία των αντικειμένων στη μνήμη.

9.2 Υλοποίηση του "Rule of Three" (Constructor, Destructor, Copy Assignment)

Αν μια κλάση διαχειρίζεται δυναμική μνήμη, **πρέπει οπωσδήποτε** να υλοποιήσουμε και τα 3 μέλη της Ορθόδοξης Κανονικής Μορφής για να εξασφαλίσουμε τη Βαθιά Αντιγραφή.

```
#include <cstring>

class DynamicString {
private:
    char* str;
public:
    // 1. Constructor (Δέσμευση μνήμης στη Heap)
    DynamicString(const char* s = "") {
        if (s == nullptr) {
            str = new char[1];
            str[0] = '\0';
        } else {
            str = new char[std::strlen(s) + 1];
            std::strcpy(str, s);
        }
    }
};
```

```

}

// 2. Destructor (Απελευθέρωση μνήμης)
~DynamicString() {
    delete[] str;
}

// 3. Copy Constructor (Κατασκευαστής Αντιγραφής - Βαθιά Αντιγραφή)
// Καλείται κατά τη δήλωση: DynamicString s2 = s1;
DynamicString(const DynamicString& other) {
    // Δέσμευση νέας μνήμης στη Heap
    str = new char[std::strlen(other.str) + 1];
    // Αντιγραφή των περιεχομένων
    std::strcpy(str, other.str);
}

// 4. Copy Assignment Operator (Τελεστής Ανάθεσης Αντιγραφής - Βαθιά Αντιγραφ
// Καλείται όταν τα αντικείμενα έχουν ήδη αρχικοποιηθεί: s2 = s1;
DynamicString& operator=(const DynamicString& other) {
    // α) Έλεγχος Αυτο-ανάθεσης (Self-assignment guard): Πρόληψη για το s1 =
    if (this == &other) {
        return *this; // Επίστρεψε αμέσως τον εαυτό σου
    }

    // β) Αποδέσμευση της παλιάς μνήμης που ήδη κρατούσε το αντικείμενο
    delete[] str;

    // γ) Δέσμευση νέας μνήμης και αντιγραφή των δεδομένων (Deep Copy)
    str = new char[std::strlen(other.str) + 1];
    std::strcpy(str, other.str);

    // δ) Επιστροφή του εαυτού μας (ως αναφορά) για chaining (s3 = s2 = s1;)
    return *this;
}
};

```

10. Διαχείριση Δυναμικής Μνήμης (new / delete)

```

// 1. Μονό Αντικείμενο
int* ptr = new int(5);
delete ptr;
ptr = nullptr;

// 2. Πίνακας (SOS: Χρήση delete[])
int* arr = new int[10];
delete[] arr;
arr = nullptr;

// 3. Αντικείμενο Κλάσης (Καλεί constructor / destructor αντίστοιχα)
MyClass* obj = new MyClass(1, 2);
delete obj;

```

11. C++ STL (Standard Template Library) Containers & Iterators

11.1 Vector (#include <vector>)

Δυναμικός συνεχόμενος πίνακας. Ιδανικός για γρήγορη ανάγνωση βάσει δείκτη.

```
#include <vector>

std::vector<int> vec;

// Βασικές Μέθοδοι:
vec.push_back(10);           // Προσθήκη στο τέλος
vec.push_back(20);           // Προσθήκη στο τέλος
vec.pop_back();               // Αφαίρεση από το τέλος (αφαιρεί το 20)

int val = vec.at(0);          // Ασφαλής ανάγνωση στοιχείου (πετάει out_of_range ex
int val2 = vec[0];            // Γρήγορη ανάγνωση (χωρίς έλεγχο ορίων)

int size = vec.size();        // Πόσα στοιχεία έχει τώρα
bool empty = vec.empty();     // true αν είναι άδειο

vec.clear();                  // Διαγράφει όλα τα στοιχεία
```

11.2 List (#include <list>)

Διπλά συνδεδεμένη λίστα. Κάθε στοιχείο δείχνει στο επόμενο και στο προηγούμενο. Δεν υποστηρίζει δείκτες (π.χ. `lst[3]`).

```
#include <list>

std::list<int> lst;

// Βασικές Μέθοδοι:
lst.push_back(10);           // Εισαγωγή στο τέλος
lst.push_front(5);           // Εισαγωγή στην αρχή (πολύ γρήγορη)

lst.pop_back();               // Αφαίρεση από το τέλος
lst.pop_front();              // Αφαίρεση από την αρχή

// Εισαγή/Διαγραφή στη μέση (απαιτεί iterator):
auto it = lst.begin();
lst.insert(it, 100);          // Εισάγει το 100 στη θέση του 'it'
lst.erase(it);                // Διαγράφει το στοιχείο στη θέση 'it'
```

11.3 Map (#include <map>)

Δομή συσχέτισης κλειδιού-τιμής (Key-Value), υλοποιημένη ως ταξινομημένο δέντρο.

```
#include <map>
#include <string>

std::map<std::string, int> ages;

// Βασικές Μέθοδοι:
ages["Nikos"] = 25;           // Εισαγωγή νέου κλειδιού ή ενημέρωση υπάρχοντος
```

```

ages.insert({"Maria", 22}); // Εναλλακτικός τρόπος εισαγωγής (μέσω std::

// Αναζήτηση στοιχείου:
auto it = ages.find("Nikos");
if (it != ages.end()) {
    // Το κλειδί βρέθηκε!
    // it->first είναι το κλειδί ("Nikos")
    // it->second είναι η τιμή (25)
    int age = it->second;
}

ages.erase("Maria"); // Διαγραφή βάσει κλειδιού

```

11.4 Iterators (Πλοηγοί) & Προσπέλαση

Ο Iterator λειτουργεί ως «γενικευμένος δείκτης».

```

std::vector<int> nums = {10, 20, 30};

// 1. Παραδοσιακός Iterator (Ανάγνωση & Γραφή)
for (std::vector<int>::iterator it = nums.begin(); it != nums.end(); ++it) {
    *it += 5; // Το '*' κάνει dereference: παίρνουμε και αλλάζουμε την τιμή
}

// 2. Const Iterator (Μόνο για Ανάγνωση / Read-Only)
for (std::vector<int>::const_iterator it = nums.begin(); it != nums.end(); ++it)
    int val = *it; // Μπορούμε μόνο να διαβάσουμε, όχι να γράψουμε (*it = 5 -> ER
}

// 3. Σύγχρονη Προσπέλαση (Range-based for - C++11)
// Ανάγνωση και Τροποποίηση (Reference &)
for (int& value : nums) {
    value += 10;
}

// Μόνο Ανάγνωση (Const Reference const&) - Πιο αποδοτικό για μεγάλα αντικείμενα
for (const int& value : nums) {
    std::cout << value << " ";
}

```

11.5 Η λέξη-κλειδί `auto` (Type Inference - C++11)

Ο compiler βρίσκει αυτόματα τον ακριβή τύπο της μεταβλητής κατά το compile-time βάσει της τιμής αρχικοποίησης (στατικός τύπος, μηδενικό runtime κόστος).

```

auto a = 5; // O compiler καταλαβαίνει int
auto b = 3.14; // O compiler καταλαβαίνει double

// 1. Σωτήριο για τεράστιους τύπους της STL:
// Αντί για: std::map<std::string, int>::const_iterator it = ages.begin();
auto it = ages.begin(); // Καθαρός και ευανάγνωστος κώδικας!

// 2. auto & const & References (Πολύ Σημαντικό!):

```

```
// Το απλό auto ΔΕΝ κρατάει const ή reference από μόνο του (κάνει copy).
int x = 10;
const int& ref = x;

auto copy = ref;           // Τύπος: int (γίνεται αντίγραφο!)
const auto& realRef = ref;  // Τύπος: const int& (πραγματική const αναφορά)
```

12. Συχνά Λάθη & Τι να Αποφεύγεις (Common Pitfalls)

- **Λάθος επιλογή `delete`**: Χρήση του `delete` αντί για `delete[]` κατά την αποδέσμευση πινάκων (`new T[]`).
- **Object Slicing (Αποκοπή)**: Πέρασμα παραμέτρου με τιμή (by value) αντί για αναφορά σε περιπτώσεις πολυμορφισμού. `void func(Base b)` (Λάθος) → `void func(const Base& b)` (Σωστό).
- **Dangling Pointer**: Δείκτης που δείχνει σε μνήμη που έγινε `delete`. Πάντα: `ptr = nullptr`; μετά τη διαγραφή.
- **Ξεχασμένο Ερωτηματικό ;** μετά από Κλάση/Δομή:

```
class MyClass {
    // ...
}; // <-- Το νούμερο 1 συντακτικό λάθος αρχαρίων!
```

Λάθη με Iterators & Maps (SOS)

- **Λάθος τύπος `std::pair` στην προσπέλαση Map**: Τα στοιχεία ενός `std::map<K, V>` αποθηκεύονται στη μνήμη ως `std::pair<const K, V>`. Προσοχή στο `const` στο Key!

```
// ΛΑΘΟΣ ( compilation error ή άσκοπα αντίγραφα ):
for (const std::pair<std::string, int>& p : ages) { ... }

// ΣΩΣΤΟ:
for (const std::pair<const std::string, int>& p : ages) { ... }
// Ή ακόμα καλύτερα με auto (που τα βρίσκει όλα μόνο του):
for (const auto& p : ages) {
    // p.first -> Το Κλειδί (Key)
    // p.second -> Η Τιμή (Value)
}
```

- **Κατά λάθος εισαγωγή στοιχείων στο Map με τον τελεστή `[]`**: Αν χρησιμοποιήσεις τον `[]` για να ελέγξεις αν υπάρχει ένα κλειδί, και αυτό δεν υπάρχει, ο τελεστής θα το δημιουργήσει εκείνη τη στιγμή με default τιμή (π.χ. `0` για integers)!

```
std::map<std::string, int> ages;
// Αν ψάχνουμε απλώς αν υπάρχει ο "Kostas":
if (ages["Kostas"] == 18) { ... } // ΛΑΘΟΣ! Αν δεν υπήρχε, μόλις δημιουργήθηκε

// ΣΩΣΤΟ (Χρήση find):
auto it = ages.find("Kostas");
```

```
if (it != ages.end() && it->second == 18) { ... }
```

- **Ακύρωση Πλοηγών (Iterator Invalidation):** Αν διαγράψεις ή προσθέσεις στοιχεία σε ένα container (π.χ. `vector`, `list`, `map`) ενώ κάνεις loop πάνω του με iterator, ο iterator ακυρώνεται και το πρόγραμμα κρασάρει με Segment Fault.

```
// ΛΑΘΟΣ (ο it καταστρέφεται μόλις γίνει erase):  
for (auto it = vec.begin(); it != vec.end(); ++it) {  
    if (*it == 5) vec.erase(it);  
}  
  
// ΣΩΣΤΟ (η erase επιστρέφει τον επόμενο έγκυρο iterator):  
for (auto it = vec.begin(); it != vec.end(); ) {  
    if (*it == 5) {  
        it = vec.erase(it); // Ανανεώνουμε τον iterator  
    } else {  
        ++it;  
    }  
}
```

- **Προσπάθεια Dereference του `.end()`:** Ο iterator `.end()` δείχνει στη θέση μετά το τελευταίο στοιχείο (φρουρός). Η ανάγνωση της τιμής του (`*vec.end()`) είναι θανάσιμο runtime σφάλμα.